

fundación
universitaria

Etapa de Contextualización – Importancia del diseño de software

Juan Diego Castañeda González

Facultad de Ingeniería de Software
Fundación Universitaria Compensar

2310–8B. Patrón de Diseño de Software

Docente. John Fredy Céspedes Beltrán

9 de octubre de 2026

Índice

Importancia del diseño de software 3

Mentefacto sobre el patrón de diseño 4

Importancia del diseño de software

El diseño en la construcción del software, es una etapa fundamental que actúa como puente entre los requerimientos y la implementación. Su importancia se debe a que permite establecer una estructura evidente en donde se define cómo se organizan, almacenan y gestionan los datos, así como la manera en que los componentes y clases de la aplicación se comunican e interactúan entre sí. A través de un buen diseño y patrón de diseño, se logra mantener la independencia entre los módulos y facilita el mantenimiento a largo plazo, ocultando la complejidad del código detrás de capas abstractas.

El desarrollo de software sin un diseño previo o una planificación arquitectónica puede generar múltiples efectos adversos.

Principalmente por parte de los desarrolladores existe el riesgo de las malas prácticas de implementación de antipatrones, siendo soluciones a problemas frecuentes que inicialmente parecen lógicas, pero que en la práctica resultan ineficaces, contraproducentes y/o generan fallos y errores dentro de la solución del programa. Sin un diseño estructurado al momento de incorporar toda la lógica en un solo lugar (como en el constructor de un objeto), el código tiende a volverse demasiado grande, poco modular y sumamente difícil de manejar. Además, cualquier modificación o adición futura obligará al equipo a alterar código original que ya funcionaba, rompiendo los principios de flexibilidad y escalabilidad.

Así mismo, los patrones de diseño son modelos y soluciones reutilizables y repetitivas a problemas comunes dentro de la interacción entre los objetos. Este concepto es crucial para cualquier desarrollador de software sin importar el rol o cargo que tenga. Los tipos de patrones de diseño como: los de creación (Singleton, Factory Method, Abstract Factory, Prototype o Builder), estructurales (Adapter) o de comportamiento (como Iterator) proporciona un lenguaje común y directrices estandarizadas.

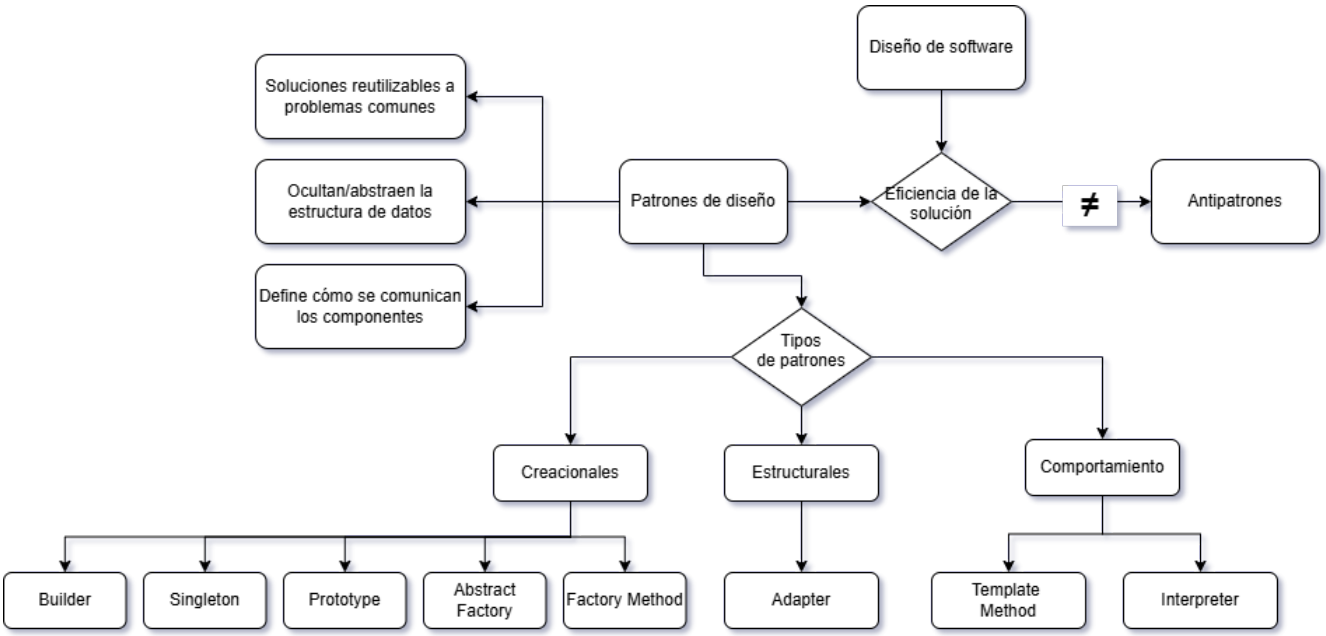
Un software fundamentado en las prácticas de un buen diseño facilita enormemente la ejecución de pruebas unitarias. Esto se debe a que un diseño modular permite aislar y probar de forma independiente los componentes más pequeños de la aplicación, comprobando que cada unidad funciona de manera correcta por sí sola.

En conclusión, el uso adecuado de los principios de diseño y sus patrones permite a todos los roles de ingeniería colaborar eficientemente en la construcción de soluciones robustas.

Mentefacto sobre el patrón de diseño

Figura 1

Mentefacto del Patron de Diseño de Software



Nota. Elaboración propia.